

Laboratorio 7

Para este laboratorio cree un repositorio de github

- Grabe un video de no más de 10 minutos donde muestre la ejecución de su programa para el ejercicio1. Súbalo a YouTube como video no listado y adjúntelo en el README. de su repositorio.
- Para los incisos que no involucren la generación de código, cree una carpeta por separado en su repositorio y coloque las respuestas en un documento con formato PDF

Problema 1: 50%

Seleccione un lenguaje de programación de alto nivel y cree un programa que realice las siguientes operaciones relacionadas a la simplificación de gramáticas:

- Su programa deberá cargar archivos de texto que contengan gramáticas. Para este laboratorio, cree dos archivos distintos, uno por cada una de las gramáticas del Ejercicio No. 2.
 - Cada línea representa una producción en su gramática. Puede tener más de una producción por línea si estas están separadas por un operador OR |.
 - Las letras individuales en mayúscula son no-terminales y las letras individuales en minúscula son terminales. Más adelante, en proyectos subsiguientes esta convención puede verse afectada por la introducción de palabras que representan a los terminales o no terminales.
- Su programa deberá interpretar correctamente que cada línea del archivo esté bien escrita, de lo contrario la ejecución debe detenerse. Para ello, utilice el trabajo empleado en el Proyecto 1 de este curso para validar que cada línea está bien escrita. Por ejemplo, cree una expresión regular que acepte producciones para gramáticas, es decir, su expresión regular debe validar que se puedan escribir expresiones como:

$$S \rightarrow 0A0 \mid 1B1 \mid BB$$

significando un texto que comience con un símbolo en mayúscula, seguido de una flecha, seguido de cuerpos de producción, que pueden estar separados por un OR.

- En el video de demostración, cambie ligeramente las producciones para introducir errores y que se vea reflejada esta validación de sus gramáticas.
- Una regex para aceptar producciones como la anterior podría verse de la siguiente manera (es un ejemplo):

$$[A-Z](\backslash n) \rightarrow ((\backslash n)([A-Za-z0-9])^*(\backslash n)^*) \mid \epsilon$$

- Luego de interpretar las producciones dentro de los archivos de texto, cree un algoritmo en su programa que elimine producciones- ϵ . Deberá mostrar en pantalla la ejecución de los pasos de su algoritmo para remover las producciones de este tipo y deberá mostrar el resultado de la gramática sin producciones- ϵ .
 - Recuerde que para remover producciones- ϵ deberá:
 - * Encontrar símbolos y producciones anulables.
 - * Encontrar las nuevas producciones de la gramática tomando en cuenta todos los 2^m casos posibles para formular producciones con base en aquellas que tengan m símbolos anulables.

Problema 2: 50%

Para las siguientes CFGs realice lo solicitado, mostrando todo el procedimiento.

1.

$$S \rightarrow 0A0 \mid 1B1 \mid BB$$

$$A \rightarrow C$$

$$B \rightarrow S \mid A$$

$$C \rightarrow S \mid \epsilon$$

2.

$$S \rightarrow aAa \mid bBb \mid \epsilon$$

$$A \rightarrow C \mid a$$

$$B \rightarrow C \mid b$$

$$C \rightarrow CDE \mid \epsilon$$

$$D \rightarrow A \mid B \mid ab$$

3.

$$S \rightarrow ASA \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \epsilon$$

- a) Elimine las producciones- ϵ .
- b) Elimine cualquier producción unaria de la gramática resultante.
- c) Elimine cualquier símbolo inútil de la gramática resultante.
 - a. Remueva símbolos que no producen.
 - b. Remueva símbolos no alcanzables.
- d) Coloque la CFG resultante en la Forma Normal de Chomsky (CNF).